



Livrable 1

Projet EduScore

Évaluation du Risque de Décrochage Scolaire

Rapport d'architecture, risques et coûts
pour le déploiement sur AWS

Groupe : 1
Plateforme : AWS, région eu-west-1
Date : 06 juin 2026

0 Table des matières

1	Présentation du projet	2
2	Schéma d'architecture prévue	2
3	Analyse de risques	3
4	Gestion des accès (IAM)	4
5	Estimation des coûts	5
6	Plan de mise en œuvre	5
6.1	Phase 0 : Fondamentaux	5
6.2	Phase 1 : Sécurité du code	5
6.3	Phase 2 : Authentification & API	6
6.4	Phase 3 : Infrastructure AWS	6
6.5	Phase 4 : Déploiement & Secrets	6
6.6	Phase 5 : Validation & Audit	6
7	Références	6

1 Présentation du projet

EduScore est une plateforme d'aide à la décision destinée aux établissements scolaires. Elle permet d'identifier rapidement les élèves susceptibles d'être en difficulté ou en risque de décrochage.

L'enseignant saisit quelques informations sur l'élève (résultats scolaires, assiduité, temps d'étude, accès aux ressources, etc.) et l'application prédit l'une des trois catégories : **En réussite**, **À surveiller** ou **À risque de décrochage**, accompagnée d'un score de confiance.

La solution s'appuie sur un modèle de classification multi-classe développé avec **scikit-learn** et déployé sur **AWS (eu-west-1)** dans une architecture conteneurisée. Ce projet met en pratique les concepts de **cloud computing**, notamment **IAM**, **VPC**, **sécurité réseau** et **ECS**.

2 Schéma d'architecture prévue

L'architecture déployée est la variante budget : les conteneurs tournent dans des sous-réseaux publics, protégés par des Security Groups robustes, sans NAT Gateway. La base de données PostgreSQL reste en sous-réseau privé avec chiffrement. Le modèle ML est stocké en S3 privé, chiffré.

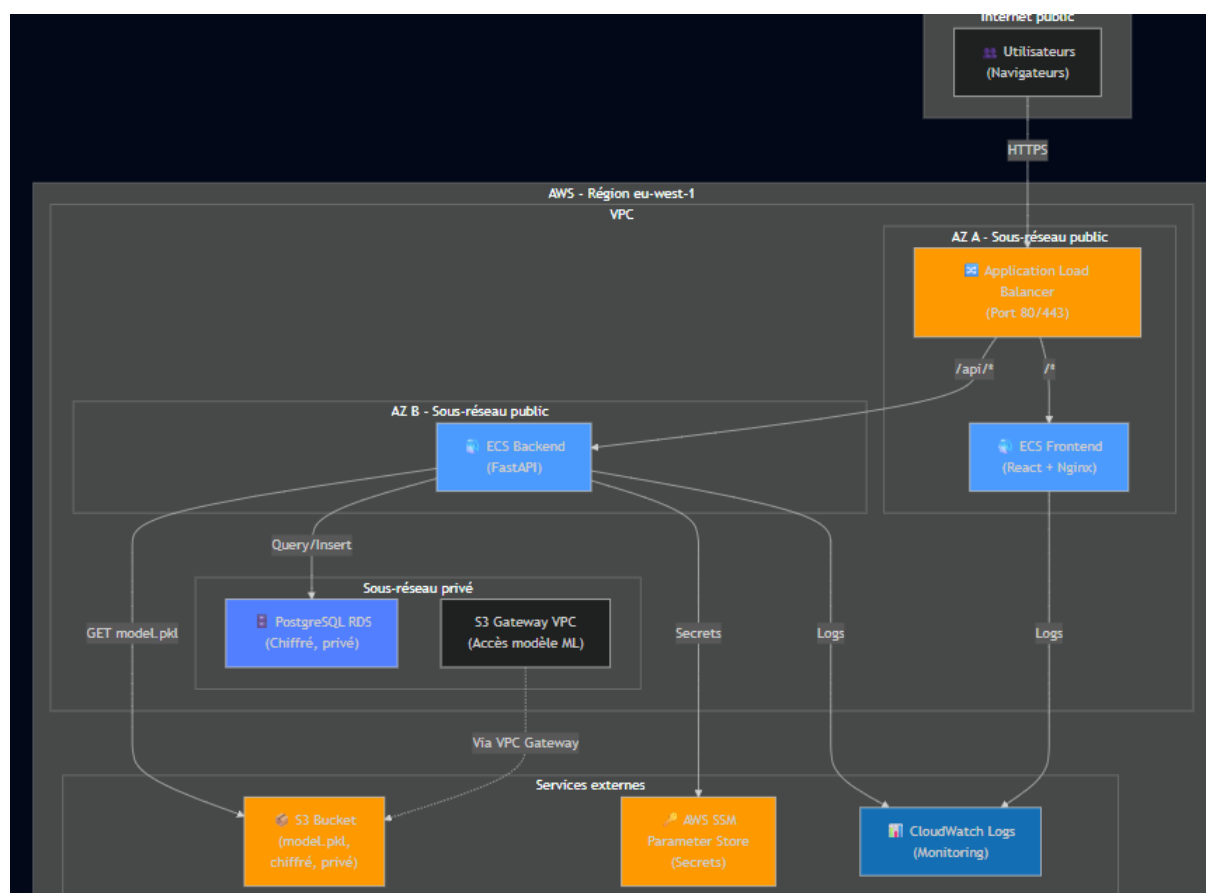


FIGURE 1 – Architecture AWS EduScore

2.0 Flux opérationnel

- Le navigateur de l'utilisateur atteint un **Application Load Balancer** public sur ports 80/443.
- L'ALB route les requêtes `/api/*` vers le service **backend ECS Fargate** (FastAPI), le reste vers le **frontend ECS Fargate** (React + nginx).
- Le backend charge le **modèle ML depuis S3** au démarrage avec cache local, stocke les prédictions dans **PostgreSQL RDS**, et récupère les secrets (JWT, Google Client ID) depuis **AWS SSM Parameter Store**.
- Le bucket S3 du modèle, la base de données, et SSM sont tous **privés** : aucun accès direct depuis Internet. L'accès se fait uniquement via les rôles IAM des tâches ECS.
- Les logs de tous les services sont centralisés dans **CloudWatch** pour audit et debugging.

2.0 Redondance et haute disponibilité

- ALB réparti sur deux zones de disponibilité (AZ A et AZ B).
- Deux tâches ECS backend/frontend déployées sur les deux AZ.
- Base PostgreSQL RDS avec backup automatique et snapshots chiffrés.
- Security Groups restreignent le trafic au strict nécessaire.

3 Analyse de risques

Risque	Impact	Prob.	Mesure prévue
Fuite de secrets (clés API, JWT_SECRET) dans Git	Élevé	Moyenne	<code>.gitignore</code> + <code>.env.example</code> ; scan Trufflehog en CI; secrets dans env.
Données d'élèves exposées (S3/RDS public)	Élevé	Moyenne	Block Public Access S3; RDS en sous-réseau privé; chiffrement S3/RDS; VPC Gateway
Accès non autorisé à l'API de prédiction	Élevé	Moyenne	Google OAuth + JWT; CORS restreint; rate-limit 20 req/min/IP sur <code>/predict</code>
Vol du modèle par requêtes massives	Moyen	Moyenne	Rate-limit strict; authentification requise; logs CloudWatch
Injection SQL (prédictions/historique)	Élevé	Faible	ORM SQLAlchemy; validation Pydantic v2; pas de SQL brut
Image Docker vulnérable en production	Moyen	Moyen	Scan Trivy en CI;
Dérive des coûts AWS	Moyen	Moyen	Sans NAT Gateway; monitoring ECS
Indisponibilité sur panne d'une zone	Moyen	Faible	Déploiement multi-AZ; ALB en failover automatique
Non-conformité RGPD	Élevé	Moyen	Audit des accès; suppression à la demande; consentement documenté

Risque	Impact	Prob.	Mesure prévue
Attaque DDoS sur le frontend public	Moyen	Faible	ALB + WAF AWS optionnel ; rate-limiting applicatif

Risque prioritaire : la confidentialité des données scolaires. Les dossiers d'élèves contiennent des données sensibles (résultats). La conformité RGPD et les directives éducatives imposent de maintenir S3 et RDS strictement privés.

4 Gestion des accès (IAM)

Rôle	Pour qui	Permissions	Principe
Administrateur	Coordinateur	Accès complet AWS : ECS, ECR, RDS, S3, SSM, CloudWatch, budget	MFA obligatoire ; audit de tous les accès
Développeur	Membres du groupe	ECS (déploiement), ECR (push images), S3 (lecture modèles), CloudWatch (logs), SSM (consultation)	Pas de droits IAM ou deletion ; clés temporaires
CI/CD (GitHub Actions)	Pipeline d'automatisation	Assumer rôle AWS OIDC, push ECR, déploiement ECS	Identité fédérée OIDC, aucune clé d'accès stockée longtemps ; JWT GitHub validé
Rôle d'exécution ECS	Services ECS backend/frontend	Pull image ECR, écriture logs CloudWatch, accès SSM Parameter Store	Rôle technique standard, moindre privilège
Rôle applicatif backend	Code Python FastAPI	<code>s3:GetObject</code> sur le seul bucket modèle, <code>rds-db:connect</code> sur RDS, <code>ssm:GetParameter</code>	Moindre privilège strict : pas de <code>S3 ListBucket</code> , pas de droit de delete
Utilisateurs métier (frontend)	Enseignants, conseillers	Authentification Google Single Sign-On (SSO)	Contrôle d'accès au niveau application, pas AWS

4.0 Gestion des secrets

- **Secrets de CI** (rôle AWS à assumer en OIDC) : dans GitHub Secrets, temporaires.
- **Secrets applicatifs** (JWT_SECRET, Google Client ID, DATABASE_URL, AWS_REGION) : chiffrés par défaut, injectés dans le conteneur au démarrage via IAM.
- **Aucun secret en clair** dans le dépôt Git, `.env.example`

5 Estimation des coûts

Poste	Service	Détails	Coût/mois
Compute : Backend	ECS Fargate	2 tâches (0,5 vCPU, 1 GB RAM) en continu	~15 \$
Compute : Frontend	ECS Fargate	2 tâches (0,25 vCPU, 512 MB RAM) en continu	~8 \$
Répartiteur	Application Load Balancer	1 ALB, ~200 req/s pic	~18 \$
Base de données	PostgreSQL RDS	db.t3.micro, 20 GB SSD, backup automatique	~12 \$
Stockage modèle	S3 (model.pkl + logs)	~500 MB, 6 000 GET/mois	~0,50 \$
Secrets & Config	SSM Parameter Store	~10 paramètres	~0 \$ (free tier)
Monitoring & Logs	CloudWatch	~50 GB/mois logs, métriques	~5 \$
Transfert de données	Data Transfer	VPC Gateway (gratuit), Internet sortant	~2 \$
Autres	ECR, Security Groups, etc.	Stockage images + gestion	~1 \$
Total estimé		Architecture budget	~61,50 \$

5.0 Économies réalisables

- **RDS à la demande** : descendre à ~5 \$/mois avec une micro RDS, ou utiliser SQLite embarqué (0 \$).
- **ECS Spot** : réduction de 70 % sur le compute, acceptable pour une plateforme éducative sans SLA critique.
- **S3 Intelligent-Tiering** : archivage automatique des logs.

6 Plan de mise en œuvre

Étapes de sécurisation à valider :

6.1 Phase 0 : Fondamentaux

- ☐ Repository : `.gitignore` complet, `.env.example` sans valeurs réelles
- ☐ Local development : Docker Compose fonctionnel, database PostgreSQL locale
- ☐ CI/CD trigger : GitHub Actions pipeline créé, scans initialisés

6.2 Phase 1 : Sécurité du code

- ☐ **Secrets scanning** : `trufflehog` scan en CI avant chaque commit
- ☐ **Dépendances** : safety check sur `requirements.txt`

6.3 Phase 2 : Authentication & API

- ☐ **Google OAuth** : bouton « Sign in with Google » sur frontend, JWT applicatif signé backend
- ☐ **Rate-limit** : slowapi configuré, 20 req/min par IP sur `/predict`
- ☐ **CORS** : liste blanche d'origines, credentials strict
- ☐ **Validation Pydantic** : tous les endpoints validés (`PredictIn`, `AdminStats`, etc.)

6.4 Phase 3 : Infrastructure AWS

- ☐ **IAM** : rôles granulaires créés (`AdminDev`)
- ☐ **VPC setup** : subnets publics/privés sur 2 AZ, NAT Gateway pour le budget optionnel
- ☐ **RDS PostgreSQL** : instance créée, chiffrement activé, backup quotidien
- ☐ **S3** : buckets modèle et logs, Block Public Access activé, versioning, chiffrement SSE-S3

6.5 Phase 4 : Déploiement & Secrets

- ☐ **ECR** : images backend/frontend pushées, scan Trivy passé
- ☐ **ECS Fargate** : tâches déployées, logs CloudWatch activés
- ☐ **SSM Parameter Store** : `JWT_SECRET`, Google Client ID, `DATABASE_URL` injectés
- ☐ **ALB** : configuré, target groups sains, HTTPS activé
- ☐ **DNS / Domaine** : Route53 pointant vers ALB

6.6 Phase 5 : Validation & Audit

- ☐ **Tests e2e** : form submission → prédiction → historique dans admin
- ☐ **Security audit** : pen test léger, vérification Block Public Access, audit IAM
- ☐ **Monitoring** : dashboards CloudWatch pour CPU, RAM, latence, erreurs

7 Références

- Architecture diagram : `infra/`
- Configuration CI/CD : `.github/workflows/`
- Secrets exemple : `.env.example`